

第四次培训： C语言入门

从Python到C：编程世界的底层语言

预计时长：40分钟：

假设你已经掌握了Python基础知识，
现在我们将带你了解C语言的基础概念与语法。
目标：了解C语言的编程环境、语法结构与编译流程

课程目标

- 理解 C 语言的定位与特点
- 搭建并熟悉 C 语言编程环境
- 能够编写并运行第一个 C 程序
- 掌握 C 的基本语法与控制结构
- 简单了解函数概念与使用
- 理解与 Python 的主要差异

为什么要学C语言？

- **计算机世界的根基**：操作系统、编译器、嵌入式系统、驱动程序几乎都由C编写
- **高效与可控**：直接操作内存、硬件，**性能远高于**解释型语言
- **理解底层原理的钥匙**：帮助你更好理解内存、指针、CPU等概念
- **现代语言的祖先**：C → C++ → Java → Python

希望通过学习C语言，初步建立“**内存**”的概念，为后续深入学习编程和计算机原理打下基础。

Python vs C：快速对比

特性	Python	C
类型系统	动态类型	静态类型
执行方式	解释执行	编译执行
语法风格	简洁易读	严谨规范
典型应用	数据分析、AI、Web	系统开发、嵌入式、算法

- 比赛时，机器人的控制程序通常有实时性要求，C语言的**高效性**和灵活性常常能带来性能优势。而Python则更适合**快速开发**和**调试**，两者结合使用效果更佳。

C语言编译流程

完整流程：

源文件(.c)

↓ 预处理器

预处理后的源文件(.i)

↓ 编译器

目标文件(.o)

↓ 链接器

可执行文件(.exe / a.out)

一步到位：

源文件(.c) -- gcc --> 可执行文件(.exe / a.out)

与 Python 不同，C 是 **编译型语言**，执行前必须编译。

C语言编译器

C语言程序需要编译器将源代码转换为可执行文件。常用编译器包括：

1 GCC / MinGW

- **GCC** (GNU Compiler Collection)
 - 跨平台开源编译器，支持 C/C++ 等多种语言
 - Linux 和 macOS 默认可用
- **MinGW** (Minimalist GNU for Windows)
 - 在 Windows 下使用 GCC 的版本，提供轻量级的 GCC 编译环境
- **特点**：开源、灵活、命令行工具，可一步到位编译和链接

2 Clang / LLVM

- **Clang**：基于 LLVM 的 C/C++/Objective-C 编译器
- **LLVM**：底层工具链，支持代码优化、生成目标文件
- **特点**：编译速度快，**报错信息清晰**，支持跨平台
- 常用于 macOS、Linux 和现代 IDE（如 VSCode、Xcode）

3 MSVC (Microsoft Visual C++)

- Windows 平台官方编译器，集成在 Visual Studio 中
- 提供完整 IDE 支持、调试器和项目管理
- **特点**：适合 Windows 开发、兼容 Microsoft 平台 API

编程环境准备

我们一般不会直接使用命令行编译C程序，而是通过集成开发环境（IDE）来简化开发流程。常用的C语言IDE包括：

1. DevC++ / Code::Blocks IDE

- **适合初学者**，集成编译器和编辑器

2. VSCode + GCC

- 灵活配置，适合多语言开发

3. CLion

- 强大的IDE，适合大型项目

4. Visual Studio：适合Windows平台【但我们的开发环境是 linux】

🌟 第一个C程序：Hello World

下面是一个简单的C语言程序，打印“Hello, World!”到控制台：

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

运行结果：

```
Hello, World!
```

📄 如何编写、编译并运行 C 程序

1. **编写代码**：使用文本编辑器（如 VSCode、Notepad++）创建一个 `.c` 文件，例如 `helloWorld.c`，并输入上述代码。
2. **编译代码**：打开终端，导航到代码所在目录，运行以下命令：

```
gcc helloWorld.c -o helloWorld
```

这会将 `helloWorld.c` 编译成可执行文件 `helloWorld`（Linux/macOS）或 `helloWorld.exe`（Windows）。

3. **运行程序**：在终端中输入以下命令运行程序：

```
./helloWorld
```

📖 程序结构讲解

- `#include <stdio.h>`
引入标准输入输出库

- `int main()`
程序的入口函数
- `{}`
语句块（代码作用范围）
- `;`
每条语句以分号结束
- `return 0;`
程序正常结束的标志

C语言基本语法

- C语言是**区分大小写**的，变量名、函数名等必须一致
- 每条语句以**分号**结束
- 使用 `{}` 来定义代码块
- 注释：
 - 单行注释： `//`
 - 多行注释： `/* ... */`



变量声明与使用

Python中变量可以直接赋值，而C语言要求**先声明变量类型**，然后才能使用。

```
#include <stdio.h>

int main() {
    int a = 5;
    float b = 3.14;
    char c = 'A';

    printf("a = %d, b = %.2f, c = %c\n", a, b, c);

    return 0;
}
```



基本数据类型

类型	含义	示例
int	整数	int a = 5;
float	单精度浮点	float x = 3.14;
double	双精度浮点	double y = 3.1415926;
char	字符	char c = 'A';

不同的数据类型占用不同的**内存空间**，影响程序的性能和行为。

注意：C语言变量声明时必须指定类型！



输入与输出

```
#include <stdio.h>

int main() {
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("You are %d years old.\n", age);
    return 0;
}
```

- %d 整数占位符
- & 取变量地址（C语言输入的关键点）



常用输入输出函数

C语言中常用的输入输出函数有：

函数	说明	示例
printf	格式化输出到控制台	printf("Value: %d\n", a);
scanf	从控制台读取格式化输入	scanf("%d", &a);

getchar() 和 putchar() 也常用于字符输入输出。其他更多的输入输出函数可以参考标准库文档或[其他教程](#)。

常用占位符

C 语言中占位符用于格式化输入输出，常见占位符如下：

占位符	说明	示例
%d	整数	int a = 5; printf("%d", a);
%f	浮点数	float x = 3.14; printf("%f", x);
%c	字符	char c = 'A'; printf("%c", c);
%s	字符串	char str[] = "Hello"; printf("%s", str);
%lf	双精度浮点数	double y = 3.14159; printf("%lf", y);

占位符的使用

输入与输出一般搭配使用占位符使用，例如：

```
printf("Hello, %s!\n", name);
```

其中 %s 是字符串占位符，name 是变量。执行的时候会将 name 的值替换到 %s 位置。

```
scanf("%d %f", &age, &height);
```

表示从输入中读取一个整数和一个浮点数，分别存储到 age 和 height 变量中。输入时需满足格式要求，例如输入 25 175.5 。

运算符

与 Python 基本一致：

运算符	说明
+ - * /	加减乘除
%	取余
++ / --	自增 / 自减；这是 C 语言特有的运算符
== != > < >= <=	比较运算
&& / / !	逻辑运算

!?! "i++" vs "++i"

```
#include <stdio.h>
```

```
int main() {  
    int i = 0;  
    printf("%d\n", i++);  
    printf("%d\n", ++i);  
    return 0;  
}
```

输出结果：

```
0 # i++ 先使用后自增  
2 # ++i 先自增后使用
```

控制结构概览

与其他编程语言类似，C语言也有多种控制结构：

- **顺序结构**：按顺序执行代码，无特殊标记
- **选择结构**：根据条件选择执行不同的代码块（如 `if`、`switch`）
- **循环结构**：重复执行某段代码（如 `for`、`while`）

C 语言的控制结构语法更严格，需要使用大括号 `{}` 明确代码块范围。

下面我们将通过与 Python 的对比，逐一介绍这些控制结构。

控制结构：if语句

```
if (x > 0) {  
    printf("Positive\n");  
} else {  
    printf("Non-positive\n");  
}
```

Python写法：

```
if x > 0:  
    print("Positive")  
else:  
    print("Non-positive")
```

for循环

C语言（初始化、条件、更新）：

```
for (int i = 0; i < 5; i++) {  
    printf("%d\n", i);  
}
```

Python：

```
for i in range(5):  
    print(i)
```

C的for循环更灵活，也更显式。

while循环

C语言：

```
int i = 0;  
while (i < 5) {  
    printf("%d\n", i++);  
}
```

Python：

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

◆ C语言函数初步

- 函数的作用：
 - 将代码封装成模块，提高可读性和复用性
 - 与 Python 函数类似，但需要显式声明返回类型和参数类型
- 基本结构：

```
#include <stdio.h>
```

```
// 函数声明
```

```
int add(int a, int b);
```

函数需要**先声明**，告诉编译器函数的返回类型和参数类型，才能使用。

```
int main() {  
    int sum = add(3, 5);  
    printf("Sum = %d\n", sum);  
    return 0;  
}  
  
// 函数定义  
int add(int a, int b) {  
    return a + b;  
}
```

结果：

Sum = 8

⚡ 要点讲解

- `int add(int a, int b) :`
 - `int` : 函数返回类型
 - `a, b` : 参数, 必须声明类型
- 调用函数: `sum = add(3, 5);`
- 与 Python 的函数相比:
 - C 需要明确类型
 - 函数必须先声明 (或定义在 `main` 前)

本课只介绍概念, 深入语法留到第二讲。

🎯 小结

- C 是 **编译型语言**, 语法更严格
- 程序从 `main()` 开始执行
- 每条语句 **必须以分号结束**
- 输入输出函数: `printf / scanf`
- 变量 **必须声明类型**

- 使用 `{}` 来定义代码块
- 简单了解函数调用与返回值

作业与练习

任务：编写一个计算器程序

输入两个整数，输出：

- 和 (Sum)
- 差 (Difference)
- 积 (Product)
- 商 (Quotient)

循环获取用户输入，直到用户选择退出（输入 'q'）。
如果除数为 0，输出 "Error: Division by zero"。

提示：

1. 使用 `scanf` 获取用户输入
2. 使用 `printf` 输出结果
3. 使用循环结构，直到用户选择退出
4. 注意处理除数为 0 的情况

实现示例：

```
#include <stdio.h>

int main() {
    int a, b;
    char choice;

    while (1) {
        printf("Enter two integers (or 'q' to quit): ");
        if (scanf(" %c", &choice) && choice == 'q') {
            break;
        }
        ungetc(choice, stdin); // 把字符放回输入流
        scanf("%d %d", &a, &b);

        printf("Sum = %d\n", a + b);
        printf("Difference = %d\n", a - b);
        printf("Product = %d\n", a * b);
        if (b != 0) {
            printf("Quotient = %.2f\n", (float)a / b);
        } else {
            printf("Error: Division by zero\n");
        }
    }

    return 0;
}
```

❏ 下一讲预告

第2讲：深入理解函数与作用域

- 自定义函数编写
- 参数传递与返回值
- 变量作用域与生命周期
- 与 Python 函数的比较

感谢聆听

- 任何问题欢迎在群里随时提问!
- 课后练习**可选**完成，**不强求**，欢迎交流讨论~

让我们用 C 的方式重新认识编程世界。